

5 Axios - Alternative à fetch()

5.1 Qu'est-ce qu'Axios ?

Information

Axios est une bibliothèque JavaScript populaire pour faire des requêtes HTTP.

Avantages par rapport à fetch() :

- Syntaxe plus simple et lisible
- Conversion JSON automatique
- Gestion erreurs HTTP automatique
- Intercepteurs de requêtes/réponses
- Timeout et annulation de requêtes

5.2 Installation

Dans le terminal de votre projet React :

Listing 14 – Installation d'Axios

```
npm install axios
```

5.3 Syntaxe de Base

```
import axios from 'axios';

// GET request
axios.get('https://api.example.com/users')
  .then(response => {
    console.log(response.data); // Données déjà parsées !
  })
  .catch(error => {
    console.error(error);
  });
```

Listing 15 – Import et utilisation basique

5.4 Comparaison fetch() vs axios

fetch()	axios
<pre> fetch(url) .then(res => { if (!res.ok) { throw new Error(' Erreur '); } return res.json(); }) .then(data => { setData(data); }) .catch(err => { setError(err.message); }); </pre>	<pre> axios.get(url) .then(res => { setData(res.data); }) .catch(err => { setError(err.message); }); </pre>
Nécessite res.json() Vérifier res.ok	Automatique Automatique

5.5 Axios avec useEffect - Pattern Complet

IMPORTANT

Pattern EXAMEN avec Axios :

```
import { useState, useEffect } from 'react';
import axios from 'axios';

function UsersListAxios() {
  const [users, setUsers] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    axios.get('https://jsonplaceholder.typicode.com/users')
      .then(response => {
        setUsers(response.data); // Déjà parsé en JSON !
        setLoading(false);
      })
      .catch(error => {
        setError(error.message);
        setLoading(false);
      });
  }, []);

  if (loading) return <p>Chargement...</p>;
  if (error) return <p>Erreur : {error}</p>;

  return (
    <ul>
      {users.map(user => (
        <li key={user.id}>{user.name}</li>
      ))}
    </ul>
  );
}
```

5.6 async/await avec Axios

```
useEffect(() => {
  const fetchUsers = async () => {
    try {
      setLoading(true);
      const response = await axios.get('https://api.example.com/users');
      setUsers(response.data);
    } catch (error) {
      setError(error.message);
    } finally {
      setLoading(false);
    }
  };

  fetchUsers();
}, []);
```

Listing 16 – Axios avec async/await

5.7 Méthodes HTTP avec Axios

```
// GET - Récupérer des données
axios.get('/users');

// POST - Créer une ressource
axios.post('/users', {
  name: 'Ahmed',
  email: 'ahmed@example.com'
});

// PUT - Mettre à jour une ressource
axios.put('/users/1', {
  name: 'Ahmed Updated'
});

// DELETE - Supprimer une ressource
axios.delete('/users/1');
```

Listing 17 – GET

5.8 Configuration Globale (Optionnel)

Créer une instance Axios avec configuration par défaut :

```
// api.js
import axios from 'axios';

const api = axios.create({
  baseURL: 'https://api.example.com',
  timeout: 5000,
  headers: {
    'Content-Type': 'application/json'
  }
});

export default api;

// Dans vos composants
import api from './api';

api.get('/users')
  .then(res => setUsers(res.data));
```

Listing 18 – Instance Axios configurée

5.9 Quand utiliser Axios vs fetch() ?

Utilisez fetch() quand :

- Projet simple, peu de requêtes API
- Pas besoin de fonctionnalités avancées
- Préférence pour APIs natives du navigateur

Utilisez Axios quand :

- Nombreuses requêtes API
- Besoin d'intercepteurs (authentification, logs)
- Syntaxe plus lisible souhaitée
- Projet à grande échelle